



A-Laberinto-ing

Este es el camino

Resumen: Crea tu propio generador de laberintos y muestra su resultado!

Versión: 2.1

Contenido

I Prólogos	2
II Instrucciones de IA	3
III Instrucciones Comunes	5
III.1 Reglas Generales	5
III.2 Makefile	5
III.3 Directrices Adicionales	6
Parte Obligatoria	7
IV.1 Resumen	7
IV.2 Uso	7
IV.3 Formato del archivo de configuración	7
IV.4 Requisitos del laberinto	8
IV.5 Formato del archivo de salida	8
Representación Visual	10
Requisitos de reutilización de código	13
Requisitos del Readme	14
Bonificaciones	16
Envío y evaluación por pares	17

Capítulo I

- Prólogos

Los laberintos han fascinado a los humanos durante miles de años. Desde el legendario Laberinto de Knossos en la mitología griega —construido por Daedalo para encerrar al Minotauro— hasta los libros de acertijos modernos y los videojuegos, los laberintos siempre han simbolizado misterio, desafío y un diseño inteligente. En la informática, la generación de laberintos es más que diversión: es una aplicación práctica de algoritmos, aleatoriedad y teoría de grafos. Algunos algoritmos famosos usados para generar laberintos —como Prim, Kruskal o el backtracker recursivo— también se utilizan en problemas del mundo real como el diseño de redes o la generación de contenido procedural. Curiosamente, los laberintos perfectos (con un único camino entre cualquier par de puntos) están directamente relacionados con los árboles de expansión en la teoría de grafos. Construir un laberinto, especialmente uno que puedas visualizar y compartir, es una excelente forma de explorar cómo las computadoras pueden crear estructura a partir del caos — y divertirse un poco en el proceso.

“Un laberinto no es un lugar para perderse, sino un camino para encontrar.”

— Anónimo

Capítulo II

Instrucciones de IA

● Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las distintas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, abórdalas siempre con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formada o de que el contenido generado es preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

● Mensaje principal

- 👉 Utiliza la IA para reducir tareas repetitivas o tediosas.
- 👉 Desarrolla habilidades de prompting, tanto de codificación como no codificación, que beneficiarán tu futura carrera.
- 👉 Aprende cómo funcionan los sistemas de IA para anticipar y evitar mejor los riesgos comunes, sesgos y cuestiones éticas.
- 👉 Continúa desarrollando habilidades técnicas y de alto rendimiento trabajando con tus compañeros.
- 👉 Solo utiliza contenido generado por IA que entiendas completamente y para el que puedas asumir responsabilidad.

● Reglas del aprendiz:

- Debes tomarte el tiempo para explorar herramientas de IA y entender cómo funcionan, para que las puedas usar de forma ética y reducir sesgos potenciales.
- Debes reflexionar sobre tu problema antes de hacer una consulta; esto te ayuda a escribir indicaciones más claras, detalladas y relevantes utilizando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier cosa generada por IA.
- Siempre busca la revisión de pares; no dependas únicamente de tu propia validación.

● Resultados de la fase:

- Desarrolla habilidades de prompting tanto generales como específicas del dominio.
- Aumenta tu productividad con un uso eficaz de las herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

● Comentarios y ejemplos:

- Con frecuencia encontrarás situaciones □ exámenes, evaluaciones y más □ donde debes demostrar una comprensión real. Prepárate, sigue desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con compañeros a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden proporcionar perspectivas alternativas y matices valiosos. Ríe? confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

Le pregunto a la IA: “¿Cómo pruebo una función de ordenar?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Mala práctica:

Pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad y fallo mi proyecto.

✓ Buenas prácticas:

Utilizo IA para ayudar a diseñar un analizador. Luego recorro la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Mala práctica:

Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no logro justificar y fallo mi proyecto.

Capítulo III

Instrucciones Comunes

III.1 Reglas Generales

- Tu proyecto debe estar escrito en Python 3.10 o posterior.
- Tu proyecto debe cumplir con el estándar de codificación flake8.
- Tus funciones deben manejar las excepciones de manera adecuada para evitar bloqueos. Usa bloques try-except para gestionar posibles errores. Prefiere los gestores de contexto para recursos como archivos o conexiones para asegurar la limpieza automática. Si tu programa falla debido a excepciones no gestionadas durante la revisión, se considerará no funcional.
- Todos los recursos (por ejemplo, manejadores de archivos, conexiones de red) deben ser gestionados correctamente para evitar fugas. Usa gestores de contexto cuando sea posible para el manejo automático.
- Tu código debe incluir anotaciones de tipo para los parámetros de las funciones, tipos de retorno y variables donde sea aplicable (usando el módulo typing). Usa mypy para la comprobación estática de tipos. Todas las funciones deben pasar mypy sin errores.
- Incluye docstrings en funciones y clases siguiendo PEP 257 (por ejemplo, estilo Google o NumPy) para documentar el propósito, los parámetros y los retornos.

III.2 Makefile

Incluye un Makefile en tu proyecto para automatizar tareas comunes. Debe contener las siguientes reglas (el lint obligatorio implica las banderas especificadas; se recomienda encarecidamente probar `-strict` para una comprobación mejorada):

- **install:** Instala las dependencias del proyecto usando `pip`, `uv`, `pipx`, o cualquier otro gestor de paquetes de tu elección.
- **run:** Ejecuta el script principal de tu proyecto (por ejemplo, mediante el intérprete de Python).
- **debug:** Ejecuta el script principal en modo depuración usando el depurador integrado de Python (por ejemplo, `pdb`).
- **clean:** Elimina archivos temporales o cachés (por ejemplo, `__pycache__`, `.mypy_cache`) para mantener el entorno del proyecto limpio.

- lint: Ejecuta los comandos `flake8 .` y `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
- lint-strict (opcional): Ejecuta los comandos `flake8 .` y `mypy . --strict`

III.3 Directrices Adicionales

- Crea programas de prueba para verificar la funcionalidad del proyecto (no enviados ni calificados). Usa marcos como `pytest` o `unittest` para pruebas unitarias, cubriendo casos límite.
- Incluye un archivo `.gitignore` para excluir artefactos de Python.
- Se recomienda usar entornos virtuales (p. ej., `venv` o `conda`) para el aislamiento de dependencias durante el desarrollo.

Si se aplican requisitos adicionales específicos del proyecto, se indicarán inmediatamente debajo de esta sección.

Capítulo IV

Parte obligatoria

IV.1 Resumen

Implementarás un generador de laberintos en Python que tome un archivo de configuración, genere un laberinto, posiblemente perfecto (con una única ruta entre la entrada y la salida), y lo escriba en un archivo usando una representación de paredes hexadecimal. También proporcionarás una representación visual del laberinto y organizarás tu código para que la lógica de generación pueda reutilizarse más tarde.

IV.2 Uso

Tu programa debe ejecutarse con el siguiente comando:

```
python3 a_maze_ing.py config.txt
```

- a_maze_ing.py es tu archivo de programa principal. Debes usar este nombre.
- config.txt es el único argumento. Es un archivo de texto plano que define las opciones de generación del laberinto. Puedes usar un nombre de archivo diferente.

Tu programa debe manejar todos los errores con elegancia: configuración inválida, archivo no encontrado, sintaxis incorrecta, parámetros de laberinto imposibles, etc. Nunca debe fallar de forma inesperada y siempre debe proporcionar un mensaje de error claro al usuario.

IV.3 Formato del archivo de configuración

El archivo de configuración debe contener un par 'KEY=VALUE' por línea.

Las líneas que empiezan por # son comentarios y deben ignorarse.

Las claves siguientes son obligatorias:

Clave	Descripción	Ejemplo
ANCHO	Ancho del laberinto (número de celdas)	ANCHO=20
ALTURA	Altura del laberinto	ALTURA=15
ENTRADA	Coordenadas de entrada (x,y)	ENTRADA=0,0
SALIDA	Coordenadas de salida (x,y)	SALIDA=19,14
ARCHIVO_SALIDA	Nombre de archivo de salida	ARCHIVO_SALIDA=maze.txt
PERFECTO	¿Es perfecto el laberinto?	PERFECTO=True

Puede agregar claves adicionales (p. ej., semilla, algoritmo, modo de visualización) si es útil.

Un archivo de configuración predeterminado debe estar disponible en su repositorio Git.

IV.4 Requisitos del laberinto

- El laberinto debe generarse al azar, pero se requiere reproducibilidad mediante una semilla.
- Cada celda del laberinto tiene entre 0 y 4 paredes, en cada punto cardinal (Norte, Este, Sur, Oeste).
- El laberinto debe ser válido, lo que significa:
 - La entrada y la salida existen y son diferentes, dentro de los límites del laberinto.
 - La estructura asegura conectividad total y ninguna celda aislada (excepto el patrón 42, ver abajo).
 - Como la entrada y la salida son celdas específicas, debe haber paredes en los bordes externos.
 - Sus datos generados deben ser coherentes: cada celda vecina debe tener la misma pared si la hay. Por ejemplo, está prohibido que una primera celda tenga una pared en el lado este y la segunda celda detrás de esa pared sin una pared en el lado oeste.
- El laberinto no puede tener áreas abiertas grandes. Los pasillos no pueden ser más anchos que 2 celdas. Por ejemplo, puedes tener áreas abiertas de 2x3 o 3x2, pero nunca 3x3.
- Cuando se representa visualmente (ver abajo), el laberinto debe contener un visible 42 dibujado por varias celdas completamente cerradas.
- Si se activa la bandera PERFECT, el laberinto debe contener exactamente un camino entre la entrada y la salida (es decir, debe ser un laberinto perfecto).



El patrón “42” puede omitirse en caso de que el tamaño del laberinto no lo permita (es demasiado pequeño). Imprima un mensaje de error en la consola en ese caso.

IV.5 Formato de Archivo de Salida

El laberinto debe escribirse en el archivo de salida usando un dígito hexadecimal por celda, donde cada dígito codifica qué paredes están cerradas:

Bit	Dirección
0 (LSB)	Norte
1	Este
2	Sur
3	Oeste

- Una pared que se cierra establece el bit a 1, abierto significa 0.

Ejemplo: 3 (binario 0011) significa que las paredes están abiertas al sur y al oeste. **O A**
(binario 1010) significa que las paredes este y oeste están cerradas.

- Las celdas se almacenan fila por fila, una fila por línea.
- Después de una línea vacía, los siguientes 3 elementos se insertan en el archivo de salida en 3 líneas:
 - las coordenadas de entrada, las coordenadas de salida y el camino válido más corto de entrada a salida, usando las cuatro letras N, E, S, W.
- Todas las líneas terminan con un `\n`.

En conjunto con su archivo de configuración específico, este archivo de salida podría ser probado automáticamente por una Moulinette. Además, se proporciona un script de validación con este tema para controlar que el archivo de salida contenga datos coherentes.

```
bash-3.2$ cat output_maze.txt
9515391539551795151151153
EBABAE812853C1412BA812812
96A8416A84545412AC4282C2A
C3A83816A9395384453A82D02
96842A852AC07AAD13A8283C2
C1296C43AAB83AA92AAB686BA
92E853968428444682AC12902
AC3814452FA83FFF82C52C42A
85684117AFC6857FAC1383D06
C53AD043AFFFAFFF856AA8143
91441294297FAFD501142C6BA
AA912AC3843FAFFF82856D52A
842A8692A92B8517C4451552A
816AC384468285293917A9542
C416928513C443A828456C3BA
91416AA92C393A82801553AAA
A81292AA814682C6A8693C6AA
A8442(6(C2C1168552C16A9542
86956951692(1455416928552
(545545456(54555545444556

1,1
19,14
SWSESWSSESSESESESESESESESESSSEESSSEENNENEE
bash-3.2$ l
```

Ejemplo de archivo de salida

Capítulo V

Representación visual

Su programa debe proporcionar una forma de mostrar el laberinto visualmente, utilizando cualquiera de:

- Renderizado ASCII de terminal, o
- Una exhibición gráfica usando la biblioteca MiniLibX (MLX).

La visualización debe mostrar claramente las paredes, la entrada, la salida y el camino de solución.

Las interacciones del usuario deben estar disponibles, al menos para las siguientes tareas:

- Regenerar un nuevo laberinto y mostrarlo.
- Mostrar/ocultar un camino mínimo válido desde la entrada hasta la salida.
- Cambiar los colores de las paredes del laberinto.
- Opcional: establecer colores específicos para mostrar el patrón 42 .

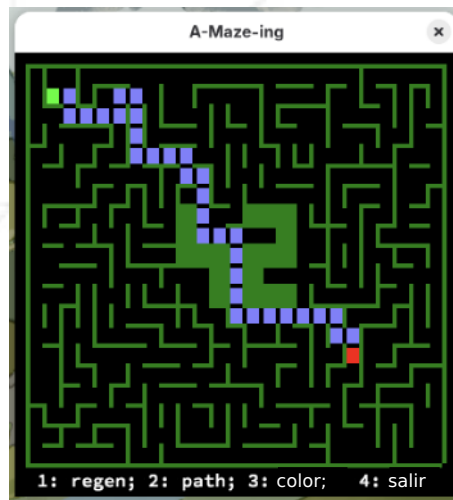
Puedes añadir interacciones de usuario adicionales.



Representación predeterminada del laberinto en la terminal



Laberintos diferentes, camino más corto y colores de las paredes



Renderizado de laberinto usando Mix

Capítulo VI

Requisitos de reutilización de código

Debes implementar la generación de laberintos como una clase única (p. ej., 'MazeGenerator') dentro de un módulo independiente que pueda ser importado en un proyecto futuro.

Debe proporcionar una breve documentación que describa cómo:

- Instanciar y usar tu generador, con al menos un ejemplo básico.
- Pasar parámetros personalizados (p. ej., tamaño, semilla).
- Acceder a la estructura generada y al menos a una solución.



El módulo generador de laberintos ofrece acceso a la estructura del laberinto, pero no necesariamente es el mismo formato que el archivo de salida.

Todo este módulo reutilizable (código y documentación) debe estar disponible en un único archivo apto para una instalación posterior con pip.

Este paquete debe llamarse mazegen-* y el archivo debe ubicarse en la raíz de tu repositorio de Git.

Se permiten las extensiones .tar.gz y .whl, como las generadas por la construcción estándar de un paquete de Python.

Ejemplo de nombre de archivo completo: mazegen-1.0.0-py3-none-any.whl.

Debes proporcionar en tu repositorio de Git todos los elementos necesarios para construir el paquete. Esto se preguntará durante la evaluación: en un virtualenv o equivalente, instala las herramientas necesarias y reconstruye tu paquete desde tus fuentes.

El archivo README.md principal (que no forma parte del módulo reutilizable) también debe contener esta breve documentación.

Capítulo VII Requisitos de Readme

Se debe proporcionar un archivo README.md en la raíz de tu repositorio de Git. Su propósito es permitir que cualquier persona no familiarizada con el proyecto (compañeros, personal, reclutadores, etc.) entienda rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El README.md debe incluir al menos:

- La primera línea debe estar en cursiva y leída: Este proyecto ha sido creado como parte del plan de estudios 42 por <login1>[, <login2>[, <login3>[...]]].
- Una sección Descripción que presente claramente el proyecto, incluyendo su objetivo y una breve visión general.
- Una sección Instrucciones que contenga cualquier información relevante sobre compilación, instalación y/o ejecución.
- Una sección Recursos que liste referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción de cómo se usó la IA — especificando para qué tareas y qué partes del proyecto.

► **Pueden requerirse secciones adicionales dependiendo del proyecto (p. ej., ejemplos de uso, lista de características, decisiones técnicas, etc.).**

Cualquier adición requerida se listará explícitamente a continuación.

- La estructura y formato completos de tu archivo de configuración.
- El algoritmo de generación de laberintos que elegiste.
- Por qué elegiste este algoritmo.
- Qué parte de tu código es reutilizable y cómo.
- Tu equipo y gestión del proyecto con:
 - Los roles de cada miembro del equipo.

- Tu planificación anticipada y cómo evolucionó hasta el final
- Lo que funcionó bien y lo que podría mejorarse
- ¿Has utilizado herramientas específicas? ¿Cuáles?

Si implementas características avanzadas (múltiples algoritmos, opciones de visualización), descríbelas en este archivo README.md.



Se recomienda inglés; alternatively, puedes usar el idioma principal de tu campus.

Capítulo

VIII Bonos

Puedes agregar varios bonos a tu proyecto. Aquí hay ejemplos posibles:

- Soportar múltiples algoritmos de generación de laberintos.
- Añadir animación durante la generación del laberinto.

Capítulo IX

Envío y evaluación por pares

Envía tu tarea en tu repositorio de Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en revisar dos veces los nombres de tus archivos para asegurarte de que sean correctos.

Durante la evaluación, ocasionalmente puede solicitarse una breve modificación del proyecto. Esto podría implicar un cambio mínimo de comportamiento, unas pocas líneas de código para escribir o reescribir, o una característica fácil de agregar.

Aunque este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar tu comprensión real de una parte específica del proyecto. La modificación puede realizarse en cualquier entorno de desarrollo que elijas (p. ej., tu configuración habitual), y debe ser factible en unos minutos, a menos que se defina un plazo específico como parte de la evaluación.

Por ejemplo, se te puede pedir hacer una pequeña actualización a una función o script, modificar una salida o ajustar una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.